

JAVA INTERVIEW PREPARATION

CHAPTER 64

ARRAYS, STRINGS, AND
SEQUENCE PATTERNS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Arrays and Strings: Reasoning Patterns for Java Interviews

Senior / Lead Java Developer Reading Chapter

Arrays are the simplest indexed data structure, yet many senior interview problems are really tests of invariant design: what a range means, when a pointer may move, what information a prefix summarizes, and whether text is being treated as bytes, UTF-16 code units, Unicode code points, or user-perceived characters.

The goal is not to memorize dozens of solutions. It is to recognize a small set of transformations and explain their correctness, complexity, and production consequences.

```
indexed sequence
-> scan once
-> maintain a bounded state or invariant
-> avoid repeatedly recomputing overlapping work
```

Java array model

Java arrays are objects created with a fixed length. Components are addressed by integer indices from `0` through `length - 1`. Primitive arrays contain primitive values; reference arrays contain references initialized to `null`.

```
int[] values = {4, 7, 9, 2};

index:    0   1   2   3
value:    [4] [7] [9] [2]
length = 4
```

Index access is constant time. Inserting at the front is linear because existing elements must move. Arrays have low per-element overhead and good locality compared with node-based structures, although the Java specification defines behavior rather than promising a particular physical layout.

An array variable is a reference. Assignment does not copy elements:

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;           // a[0] is also 99

int[] copy = Arrays.copyOf(a, a.length);
```

Multidimensional Java arrays are arrays of array references. Rows can have different lengths and can be `null`.

Covariance and runtime store checks

Reference arrays are covariant: a `String[]` can be assigned to an `Object[]`. This is type-safe only because each store is checked at runtime.

```
String[] names = new String[2];
Object[] objects = names;
objects[0] = "Ada";
objects[1] = 42;           // ArrayStoreException
```

Generics are invariant and reject analogous misuse at compile time. This distinction is a common interview topic and a reason to prefer generic collection APIs when runtime array type is not required.

Primitive arrays are not covariant with `Object[]`; `int[]` is an object, but its components are primitive values.

Range conventions prevent off-by-one errors

Choose one range convention and write it down. Java library APIs often use half-open ranges `[from, to)`: include `from`, exclude `to`.

```
[left, right)
length = right - left
empty when left == right
whole array = [0, n)
```

Half-open ranges compose cleanly: `[0, mid)` and `[mid, n)` do not overlap or leave a gap. Inclusive `[left, right]` ranges can also work, but their length is `right - left + 1`. Many bugs come from switching conventions inside one algorithm.

Before coding, define:

- what each pointer indexes;
- whether the range includes the endpoints;
- what is known before and after each iteration;
- the termination condition;
- behavior for empty input and one element.

In-place operations and ownership

An in-place algorithm uses the input array as working storage. It can reduce additional memory from $O(n)$ to $O(1)$, but mutates caller-visible state.

```
static void reverse(int[] a) {
    for (int left = 0, right = a.length - 1; left < right; left++, right--) {
        int tmp = a[left];
        a[left] = a[right];
        a[right] = tmp;
    }
}
```

The invariant is that elements outside `[left, right]` are already in their final reversed positions. Each swap shrinks the unresolved range.

Production APIs should state whether they retain or mutate arrays. Defensive copying protects ownership but costs $O(n)$ time and memory. The correct choice depends on trust boundary, size, and performance evidence.

Two pointers: opposite directions

Two pointers replace nested search when ordered movement can eliminate candidates. In a sorted array, the two-sum problem illustrates this:

```
static int[] twoSumSorted(int[] a, int target) {
    int left = 0;
    int right = a.length - 1;
    while (left < right) {
        long sum = (long) a[left] + a[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

Why is movement safe? If the current sum is too small, pairing the same smallest value with any smaller right value cannot help, so `left` can advance. Sorting supplies the monotonic property. The cast to `long` prevents integer overflow from corrupting comparison.

Complexity is $O(n)$ time and $O(1)$ extra space after sorting. If sorting is not already guaranteed, sorting adds $O(n \log n)$ and changes original positions; a hash map may be preferable.

Two pointers: same direction

A slow pointer can mark the next output position while a fast pointer scans input. Removing duplicates from a sorted array is the canonical example:

```
static int deduplicateSorted(int[] a) {
    if (a.length == 0) return 0;
    int write = 1;
    for (int read = 1; read < a.length; read++) {
        if (a[read] != a[write - 1]) {
            a[write++] = a[read];
        }
    }
    return write;
}
```

The prefix `[0, write)` always contains the unique values from the processed input. The remaining suffix is unspecified. This pattern appears in compaction, filtering, partitioning, and merging.

Fixed-size sliding windows

A sliding window reuses work between overlapping ranges. To find the maximum sum of `k` consecutive values, compute the first window once, then subtract the value leaving and add the value entering.

```
static long maxWindowSum(int[] a, int k) {
    if (k <= 0 || k > a.length) throw new IllegalArgumentException();
    long sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    long best = sum;
    for (int right = k; right < a.length; right++) {
        sum += a[right] - a[right - k];
        best = Math.max(best, sum);
    }
    return best;
}
```

The window is always exactly `k` elements. Runtime falls from $O(nk)$ repeated summation to $O(n)$, with constant extra space.

Use `long` for aggregate sums when `int` input count and magnitude can overflow. Complexity analysis that ignores numeric correctness is incomplete.

Variable-size sliding windows

Variable windows work when moving one boundary changes validity monotonically. For the longest substring without repeated code units:

```
static int longestUniqueUtf16Window(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int left = 0;
    int best = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        Integer previous = last.put(c, right);
        if (previous != null && previous >= left) left = previous + 1;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

The invariant is that `[left, right]` contains no repeated `char` value. `left` never moves backward, so each position is processed a constant number of times.

This version operates on UTF-16 code units, not full Unicode code points. For arbitrary user text, convert or iterate code points and define whether length means code points or grapheme clusters.

Sliding-window logic fails when removing a left element does not predictably restore validity. Negative numbers, for example, break some "shrink while sum is too large" assumptions because removing a negative can increase the sum.

Prefix sums

A prefix array stores cumulative information so repeated range queries become constant time.

```
values: [3, 1, 4, 2]
prefix: [0, 3, 4, 8, 10]

sum of values[left..right) = prefix[right] - prefix[left]
```

```
static long[] prefixSums(int[] a) {
    long[] prefix = new long[a.length + 1];
    for (int i = 0; i < a.length; i++) {
        prefix[i + 1] = prefix[i] + a[i];
    }
    return prefix;
}
```

Building costs $O(n)$ time and space; each range-sum query costs $O(1)$. The leading zero makes the formula uniform for ranges beginning at zero.

Prefix techniques extend beyond sums: XOR, counts by category, cumulative frequencies, and two-dimensional image or matrix regions. The combined operation must support removing a prefix contribution; minimum does not have a simple inverse.

Prefix sums with hash maps

To count subarrays whose sum equals `target`, observe that a subarray from `i` to `j` has target sum when `prefix[j] - prefix[i] == target`.

```
static long countTargetSubarrays(int[] a, long target) {
    Map<Long, Long> seen = new HashMap<>();
    seen.put(0L, 1L);
    long prefix = 0;
    long count = 0;
    for (int value : a) {
        prefix += value;
        count += seen.getOrDefault(prefix - target, 0L);
        seen.merge(prefix, 1L, Long::sum);
    }
    return count;
}
```

The initial zero prefix counts subarrays beginning at index zero. Unlike a sum-based sliding window, this works with negative numbers because it does not depend on monotonic movement.

Difference arrays for range updates

When many operations add a value across a range, store changes at boundaries rather than touching every element.

```
add +5 to [left, right)
difference[left] += 5
difference[right] -= 5

one prefix pass reconstructs final values
```

This changes q range updates from $O(qn)$ worst case to $O(q + n)$, provided final values are needed after all updates. It is unsuitable when arbitrary intermediate queries must be answered; a Fenwick or segment tree may be appropriate later.

Intervals and sorting

Interval problems become manageable after defining boundary semantics and sorting. To merge overlapping half-open intervals:

```
record Interval(int start, int end) {}

static List<Interval> merge(List<Interval> input) {
    List<Interval> sorted = input.stream()
        .sorted(Comparator.comparingInt(Interval::start))
        .toList();
    List<Interval> out = new ArrayList<>();
    for (Interval next : sorted) {
        if (out.isEmpty() || out.get(out.size() - 1).end() < next.start()) {
            out.add(next);
        } else {
            Interval last = out.remove(out.size() - 1);
            out.add(new Interval(last.start(), Math.max(last.end(), next.end())));
        }
    }
    return out;
}
```

Decide whether touching intervals merge. For half-open bookings, $[1, 3)$ and $[3, 5)$ do not overlap, though a business rule may still combine them. Validate `start <= end` and guard comparator arithmetic from overflow by using comparator methods rather than subtraction.

Arrays as frequency tables

When keys come from a small known domain, an array can replace hashing:

```
int[] counts = new int[26];
for (char c : asciiLowercaseInput.toCharArray()) {
    if (c < 'a' || c > 'z') throw new IllegalArgumentException();
    counts[c - 'a']++;
}
```

This is fast and compact only because the input contract is ASCII lowercase. Applying it to arbitrary Unicode is incorrect. A map or code-point-aware structure is needed when the domain is large or sparse.

String, char, code point, and grapheme

Java `String` uses UTF-16 semantics at the API level. `length()` counts `char` code units. Supplementary Unicode code points use a surrogate pair and therefore contribute two to `length()`.

```
String text = "A\uD83D\uDE00"; // A plus a supplementary character
int codeUnits = text.length();
int codePoints = text.codePointCount(0, text.length());
int[] points = text.codePoints().toArray();
```

A code point still does not always equal a user-perceived character. Combining marks, emoji sequences, and regional indicators can form grapheme clusters. Requirements must define what "character," reversal, truncation, and length mean.

Never split security identifiers, names, or messages by `char` without validating the domain. Normalization and locale-sensitive case conversion are separate concerns; use explicit locale and security guidance rather than ad hoc lowercase comparison.

String immutability and construction

Strings are immutable, so repeated concatenation in a loop can create many intermediate objects.

```
StringBuilder builder = new StringBuilder(estimatedSize);
for (String part : parts) {
    builder.append(part);
}
String result = builder.toString();
```

Modern compilers optimize simple concatenation expressions, but algorithmic repeated rebuilding can remain costly. Measure allocation and choose streaming when output is very large.

Substring operations should be treated as creating a value whose storage behavior is implementation-dependent. Do not rely on historical backing-array sharing. Bound retained input and copy deliberately at trust boundaries if a small token would otherwise retain a large application object through your own data model.

Matrix traversal and boundary discipline

For rectangular matrices, validate dimensions before using `matrix[0].length`. Java permits empty and ragged arrays. Spiral and grid algorithms become safer when boundaries have explicit meanings:

```
top inclusive, bottom exclusive
left inclusive, right exclusive
```

After traversing one side, move its boundary inward and check that the remaining region is nonempty before traversing the opposite side. For graph-like grid traversal, avoid recursive depth proportional to cell count when stack overflow is possible; Chapter 69 will cover traversal structures deeply.

Choosing the pattern

ordered candidate elimination	-> two pointers
contiguous range, local validity	-> sliding window
many immutable range queries	-> prefix aggregate
many offline range updates	-> difference array
small dense key domain	-> frequency array
overlapping ranges	-> sort and merge intervals
arbitrary signs and target sums	-> prefix sum plus map

State the precondition. Two pointers may require sorting; a variable window may require monotonic validity; a frequency array may require a tiny domain. The pattern is correct only when its invariant is correct.

Testing sequence algorithms

Test empty, one-element, all-equal, already ordered, reverse ordered, duplicates, negative values, integer extremes, and maximum allowed size. For strings, test empty, ASCII, BMP non-ASCII, supplementary code points, combining marks, invalid or unexpected input, and locale-sensitive cases where relevant.

Property tests are powerful:

- reversing twice returns the original;
- merged intervals are ordered and nonoverlapping;
- prefix range answers equal a simple loop oracle;
- deduplication output is sorted, unique, and drawn from input;
- optimized target-subarray counts equal $O(n^2)$ brute force on small random arrays.

Use the slow simple solution as an oracle during development, not in production.

Production connections

Arrays and these patterns appear in time-series windows, log analytics, rate-limit counters, image processing, financial aggregates, telemetry, scheduling, compression, parsers, and batch transformations. Production concerns add bounded input, numeric overflow, ownership, allocation, vectorization opportunities, and observability.

A theoretically $O(n)$ algorithm can still fail when n is unbounded or each element triggers remote I/O. Separate in-memory algorithm complexity from system interactions.

Common senior-level traps

"Array access is always safe because Java checks bounds." It fails safely with an exception, but the algorithm is still wrong and may expose a 500.

"Two pointers always reduce nested loops." Only a monotonic elimination property makes pointer movement safe.

"Sliding windows work for every contiguous sum." Negative values can break shrink/expand monotonicity.

"String length is the number of characters." It counts UTF-16 code units.

"In-place means no memory cost." The input is mutated, stack frames and returned objects still exist, and ownership matters.

"Use `int` because inputs are `int`." Aggregates and comparator subtraction can overflow.

A strong interview answer

I start by defining the sequence domain, range convention, mutation permission, numeric bounds, and for text whether units are bytes, UTF-16 code units, code points, or grapheme clusters. I write the loop invariant before choosing a pattern. Ordered candidate elimination suggests two pointers; contiguous monotonic validity suggests a sliding window; repeated immutable range queries suggest prefix aggregates; offline range updates suggest a difference array; and target sums with negative values often suggest prefix state plus a hash map. I use `long` for risky aggregates, state sorting or domain preconditions, test edge and Unicode cases, and explain both asymptotic cost and ownership or allocation consequences.

Chapter summary

Arrays make data directly indexable; the major interview patterns avoid recomputing overlapping information. Correctness comes from ranges and invariants, not pattern names. Strings add a second requirement: choose the correct textual unit before applying an array algorithm.

Revision Notes

- Java arrays are fixed-length objects with indices `0` through `length - 1`.
- Assignment aliases an array; copying requires an explicit operation.
- Reference-array covariance can fail at runtime with `ArrayStoreException`.
- Half-open ranges `[left, right)` have length `right - left`.
- In-place algorithms trade extra memory for caller-visible mutation.
- Two pointers require a safe candidate-elimination invariant.
- Slow/fast pointers compact or filter while preserving a processed prefix.
- Fixed windows reuse one aggregate across equal-length ranges.
- Variable windows require validity that changes monotonically with boundaries.
- Prefix sums answer range sums as `prefix[right] - prefix[left]`.
- Prefix plus a frequency map handles target sums with negative values.
- Difference arrays make many offline range updates efficient.
- Define touching and boundary semantics before merging intervals.
- Frequency arrays require a small validated key domain.
- `String.length()` counts UTF-16 code units, not Unicode code points.
- Code points still do not equal grapheme clusters in all user text.
- Use `StringBuilder` for deliberate repeated construction and bound large output.
- Java matrices can be empty, ragged, or contain null rows.
- Use `long` for aggregates that may exceed `int`.
- Test optimized algorithms against a simple oracle on small randomized inputs.

Likely interview question: When does a sliding window fail for a target-sum problem?

A conventional shrink-and-expand window depends on monotonic behavior. With nonnegative values, extending the window cannot reduce its sum and removing the left value cannot increase it, so boundaries can move forward without missing a candidate. Negative values destroy that reasoning: adding a value may lower the sum and removing one may raise it. For arbitrary signed values, prefix sums plus a map of previously seen prefix frequencies is often the correct linear-time technique.

Likely interview question: What is the difference between a code unit, code point, and visible character?

Java `char` and `String.length()` operate in UTF-16 code units. A Unicode code point may occupy one code unit or a surrogate pair of two. A user-perceived character, or grapheme cluster, can contain multiple code points, such as a base letter plus combining marks or a joined emoji sequence. The algorithm must select the unit that matches the product requirement; code-point iteration alone does not solve every display-text problem.

Likely interview question: Why use half-open ranges?

For `[left, right)`, length is always `right - left`, an empty range has equal boundaries, adjacent ranges meet without overlap, and prefix range sums become `prefix[right] - prefix[left]`. The convention does not prevent every bug, but it makes boundary algebra consistent across loops, slices, and windows.

Likely interview question: Does $O(n)$ guarantee good production performance?

No. Constant work, allocation, locality, bounds, and I/O still matter. An $O(n)$ operation over an unbounded request can violate latency or memory limits. Connect complexity to maximum size.